Software Requirements Specification

for

SKKU Eats

Prepared by 김지수, 김정욱, 양현준, 이동균, 장훈, 정민석

Introduction to Software Engineering Team 03

2025.11.11

Table of Contents

Table of Contents

List of Tables

List of Pictures

1. Introduction
- 1.1 Purpose
- 1.2 Product Scope
- 1.3 Definitions, Acronyms and Abbreviation
- 1.4 Overview
- 1.5 References

2. Overall Description
- 2.1 Product Perspective
  - 2.1.1 Market Status
  - 2.1.2 Overall Structure
- 2.2 Product Functions
  - 2.2.1 로그인 및 사용자 관리
  - 2.2.2 메인 페이지 및 식당 정보 탐색
  - 2.2.3 메뉴 선택 및 장바구니
  - 2.2.4 결제 및 QR 코드 발급
  - 2.2.5 리뷰 및 피드백
  - 2.2.6 즐겨찾기 및 최근 주문 관리
- 2.3 User Classes and Characteristics
  - 2.3.1 User
  - 2.3.2 System Administrator
- 2.4 Operating Environment
  - 2.4.1 Hardware
  - 2.4.2 Software
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. Specific Requirements
- 3.1 External Interface Requirements
  - 3.1.1 User Interface
  - 3.1.2 Hardware Interfaces
  - 3.1.3 Software Interfaces
  - 3.1.4 Communications Interfaces
- 3.2 Functional Requirements
  - 3.2.1 Use Case
  - 3.2.2 Use Case Diagram
  - 3.2.3 Data Dictionary(RDB)
- 3.3 Non-Functional Requirements
  - 3.3.1 Product Requirements
    - 3.3.1.1 Usability Requirements
    - 3.3.1.2 Efficiency Requirements
    - 3.3.1.3 Dependability Requirements

3.3.1.4 Security Requirements
3.3.2 Organization Requirements
3.3.2.1 Environmental Requirements
3.3.2.2 Development Requirements
3.3.3 External Requirements
3.3.3.1 Regulatory Requirements
3.3.3.2 Ethical Requirements
3.3.4 Organizing the Specific Requirements
3.3.4.1 Context Model
3.4 API Specification
3.5 Server Architecture
3.6 Infrastructure Architecture
3.6.1 Overview
3.6.2 애플리케이션 배포
3.6.3 인프라 운영
3.7 Project Management
3.8 System Evolution
3.8.1 Limitation and Assumption
3.8.2 Anticipated Changes
4. Additional Information
4.1 Format

List of Tables

[표 1] Definitions
[표 2] Abbreviations and Acronyms
[표 3] Interface of Login page
[표 4] Interface of Main page
[표 5] Interface of Real-time Crowding page
[표 6] Interface of Payment History page
[표 7] Interface of Favorites page
[표 8] Interface of Restaurant Menu page
[표 9] Interface of Menu Detail page
[표 10] Interface of Cart page
[표 11] Interface of Payment page
[표 12] Interface of QR page
[표 13] Interface of Review Writing page
[표 14] Hardware Interface
[표 15] Software Interface
[표 16] Communication Interface
[표 17] Use Case of Login and Authentication
[표 18] Use Case of Entering Main Page
[표 19] Use Case of Viewing Restaurant Information
[표 20] Use Case of Menu Selection and Ordering
[표 21] Use Case of Reviewing Cart
[표 22] Use Case of Payment
[표 23] Use Case of Checking QR

[표 24] Use Case of Review Submission
[표 25] User Data Dictionary
[표 26] Restaurant Data Dictionary
[표 27] Menu Data Dictionary
[표 28] Order Data Dictionary
[표 29] Review Data Dictionary
[표 30] Favorite Data Dictionary
[표 31] 로그인 API의 파라미터 및 응답 형식 예시
[표 32] 혼잡도 조회 API의 파라미터 및 응답 형식 예시
[표 33] 식단 조회 API의 파라미터 및 응답 형식 예시
[표 34] 리뷰 작성 API의 파라미터 및 응답 형식 예시
[표 35] 결제 API의 파라미터 및 응답 형식 예시

List of Pictures

[그림 1] Overall Structure
[그림 2] Use-Case Diagram
[그림 3] RDB ERD
[그림 4] System Context Model
[그림 5] 서버 구조
[그림 6] 인프라 아키텍쳐
[그림 7] 애플리케이션 배포 과정에 대한 상세 시퀀스 다이어그램
[그림 8] 노션으로 구축한 칸반보드 예시
[그림 9] 코드 관리 관련 브랜치

1. Introduction

1.1 Purpose

본 문서는 'SKKU Eats' 애플리케이션 서비스를 제공하기 위한 SRS(Software Requirements Specification)이다. SKKU Eats는 성균관대학교 학내 식당 이용자의 편의를 향상하기 위해 개발된 학식 통합 플랫폼으로, 메뉴 조회, 주문, 결제, 리뷰, 혼잡도 확인 기능을 하나의 플랫폼에서 제공한다. 이 문서는 해당 서비스의 요구사항을 명확히 정의하고, 본 내용을 기반으로 개발자, 디자이너, 테스트 담당자 등 프로젝트 이해관계자들이 동일한 방향성을 가지고 시스템을 설계하고 구현할 수 있도록 하는 것을 목적으로 한다. 본 애플리케이션은 교내 여러 식당의 실시간 혼잡도 정보 제공, 모바일 결제 및 QR을 통한 수령 기능, 리뷰 기반 피드백 시스템을 통합하여 사용자의 대기 시간을 단축하고 효율적인 학식 환경을 제공하는 것을 목표로 한다. 이 SRS는 이러한 기능적·비기능적 요구사항을 명세하고, 애플리케이션의 개발 범위와 주요 목표를 구체적으로 제시함으로써 향후 유지보수 및 기능 확장 시에도 일관된 품질과 방향성을 유지할 수 있도록 지원한다.

1.2 Product Scope

성균관대학교의 학생들은 교내 식당을 이용하면서 여러 가지 불편함을 겪고 있다. 대표적으로 긴 키오스크 대기 줄 병목현상으로 인한 시간 낭비, 예상치 못한 품절 문제, 그리고 메뉴에 대한 신뢰할 만한 리뷰의 부족이 있다. 실제로 학내 커뮤니티에서는 "점심시간에 줄이 너무 길다", "특식이 일찍 동나서 먹지 못했다"와 같은 불만이 꾸준히 제기되어 왔다. 이러한 문제는 단순한 불편을 넘어, 짧은 휴식 시간을 가진 학생들의 식사 효율을 떨어뜨리고, 전체적인 만족도를 낮추는 주요 원인으로 작용하고 있다.

이러한 상황을 해결하기 위해 SKKU Eats는 성균관대학교 학내 식당의 이용 과정을 디지털화하고 효율화하는 것을 목표로 한다. 본 서비스는 사용자가 한눈에 식당의 실시간 혼잡도와 재고 현황을 확인할 수 있도록 하며, 이를 통해 불필요한 대기 시간을 줄이고 원하는 메뉴를 적시에 선택할 수 있도록 돕는다. 또한, 모바일 주문과 결제 기능을 도입함으로써 기존의 키오스크 현장 결제 방식을 대체하고, 줄을 서지 않고도 간편하게 식사를 수령할 수 있는 환경을 제공한다. 이와 더불어, 메뉴 리뷰 작성 및 열람 기능을 통해 다른 이용자들의 평가를 공유하고 참고할 수 있게 하여, 식사 선택의 만족도를 높인다.

이 서비스는 단순한 주문·결제 애플리케이션이 아니라, 학내 식당 운영의 효율성을 높이는 데이터 기반 플랫폼이라는 점에서 의미가 있다. 학생들은 '혼잡도-재고-리뷰'의 세 가지 정보를 기반으로 합리적인 선택을 할 수 있고, 식당 운영자는 사용자 데이터를 분석해 인기 메뉴나 피크 시간대를 파악함으로써 보다 효율적인 운영이 가능하다. 또한 즐겨찾기 기능을 통해 자주 방문하는 식당이나 메뉴를 저장함으로써, 반복적인 주문 절차를 최소화하고 사용자의 편의성을 한층 강화한다.

결과적으로 SKKU Eats는 성균관대학교 학생들의 식사 경험을 개선하고, 학내 식당의 디지털 전환을 촉진하는 것을 목표로 한다. 본 애플리케이션은 단순한 식사 보조 도구를 넘어, "학내 구성원의 시간을 절약하고, 더 나은 선택을 돕는 스마트 식사 관리 플랫폼"으로 자리 잡는 것을 지향한다.

1.3 Definitions, Acronyms and Abbreviation

아래 표는 본 문서(SRS) 내에서 사용된 주요 용어 및 약어에 대한 정의를 나타낸다.

약어	설명
User (사용자)	'SKKU Eats' 애플리케이션을 사용하는 성균관대학교 학생 또는 교직원
System Administrator (시스템 관리자)	사용자 계정 관리, 식당 정보 및 메뉴 데이터 등록·수정·삭제를 담당하는 관리자
Client (클라이언트)	서버와 통신하여 서비스를 요청하고 결과를 수신하는 사용자 단말(모바일 기기)
Server (서버)	사용자 요청을 처리하고 데이터베이스와 상호작용하여 정보를 제공하는 중앙 시스템
Software (소프트웨어)	'SKKU Eats' 서비스를 구성하는 전체 애플리케이션 프로그램 및 관련 모듈
DB (Database)	식당, 메뉴, 재고, 결제, 리뷰 등 서비스 전반의 데이터를 저장하고 관리하는 시스템

[표 1] Definitions

약어	설명
OS	Operating System, 애플리케이션이 구동되는

	기본 시스템 환경
Backend	사용자에게 직접 노출되지 않으며, 서버 로직과 데이터 처리를 담당하는 시스템 구성 요소
Frontend	사용자가 직접 조작하고 시각적으로 확인할 수 있는 애플리케이션 화면 영역
UI/UX	User Interface / User Experience, 사용자 인터페이스 및 경험을 의미
API	Application Programming Interface, 클라이언트와 서버 간의 통신을 위한 인터페이스
HTTP	Hypertext Transfer Protocol, 웹 통신을 위한 표준 전송 프로토콜
QR	Quick Response Code, 결제 후 식사 수령을 위한 고유 식별 코드
PK	Primary Key, 데이터베이스에서 각 레코드를 구분하기 위한 고유 키
FK	Foreign Key, 다른 테이블과의 관계를 정의하기 위한 외래 키

[표 2] Abbreviations and Acronyms

1.4 Overview

본 소프트웨어 요구사항 명세서는 총 3개의 chapter와 부록으로 구성되어 있다. Chapter 1에서는 SKKU Eats 애플리케이션의 목적과 개발 배경, 그리고 프로젝트의 전반적인 개요를 설명한다. 이 장에서는 애플리케이션의 개발 계기와 주요 기능, 기대 효과를 포함하여 본 문서가 다루는 전체 시스템의 방향성을 제시한다. Chapter 2에서는 SKKU Eats의 전체 구조와 동작 환경, 그리고 시스템이 구현될 실제 사용 맥락을 설명한다. 사용자 관점에서의 서비스 이용 흐름과, 관리자 및 서버 간의 상호작용 관계를 구체적으로 기술한다. 또한 애플리케이션의 기능이 수행되는 기반 환경(모바일 클라이언트, 백엔드 서버, 데이터베이스 등)과 시스템 간 의존성을 명확히 정의한다. Chapter 3에서는 본 프로젝트의 구체적인 요구사항을 다룬다. 이 장에서는 기능적 요구사항과 비기능적 요구사항을 구분하여 명시하며, 각 기능의 입력, 처리, 출력 과정을 세부적으로 기술한다. 또한 인터페이스 요구사항, 제약 조건, 그리고 향후 유지보수 시 고려해야 할 사항들을 포함하여 시스템 전체의 일관된 품질 기준을 제시한다. 마지막으로 본 문서는 SKKU

Eats의 설계 및 구현 과정에서 모든 개발자, 디자이너, 그리고 테스트 담당자가 참조할 수 있는 표준 지침으로 사용되며, 이후 수정·보완 시 해당 내용과 담당자를 명확히 기록하여 관리한다.

1.5 References

- IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications, In IEEEXplore Digital Library http://ieeexplore.ieee.org/Xplore/guesthome.jsp
- "Software Requirement Specification". SKKU 2024 Spring Introduction to Software Engineering Team11. https://github.com/skkusal/SWE3002_42/blob/main/2024s/Team11/Team11_SRS.pdf

2. Overall Description

2.1 Product Perspective

본 애플리케이션은 성균관대학교 학내 구성원을 위한 식당 통합 관리 서비스 소프트웨어로, 사용자가 캠퍼스 내 여러 식당의 정보를 손쉽게 확인하고, 메뉴 주문 및 결제 과정을 효율적으로 수행할 수 있도록 설계되었다. SKKU Eats는 기존 오프라인 중심의 식당 이용 방식에서 발생하는 불편함인 긴 키오스크 대기 줄, 재고 부족, 메뉴 정보 부재 등을 해소하고자 개발되었다.

본 서비스는 사용자가 식당을 선택하고 메뉴를 탐색하는 단계부터 결제 완료 후 QR 코드로 수령을 진행하는 전 과정이 하나의 애플리케이션 내에서 이루어지도록 통합되어 있다. 이를 통해 사용자는 각 식당의 실시간 재고 현황과 혼잡도 정보를 한눈에 확인할 수 있으며, 결제 시에는 앱 내 결제 기능을 활용하여 대기 시간을 단축할 수 있다.

또한, SKKU Eats는 단순한 식당 안내 서비스가 아니라, 사용자 즐겨찾기 및 리뷰 기반 피드백 시스템을 포함한다. 사용자는 즐겨찾기를 통해 자주 찾는 메뉴를 빠르게 선택할 수 있으며, 식사 후 직접 사진을 포함한 리뷰를 작성하여 다른 사용자에게 유용한 정보를 제공할 수 있다. 이와 같은 기능들은 사용자의 편의성을 높이는 동시에, 학교 식당 운영진이 실시간으로 이용 현황을 분석하고 효율적인 재고 관리 및 메뉴 개선에 반영할 수 있도록 지원한다.

결과적으로, 본 애플리케이션은 성균관대학교 캠퍼스 내 식당 이용의 편의성, 신속성, 투명성을 향상하는 것을 목표로 하며, 학내 구성원 모두가 효율적인 식사 경험을 누릴 수 있도록 하는 데 그 의의가 있다.

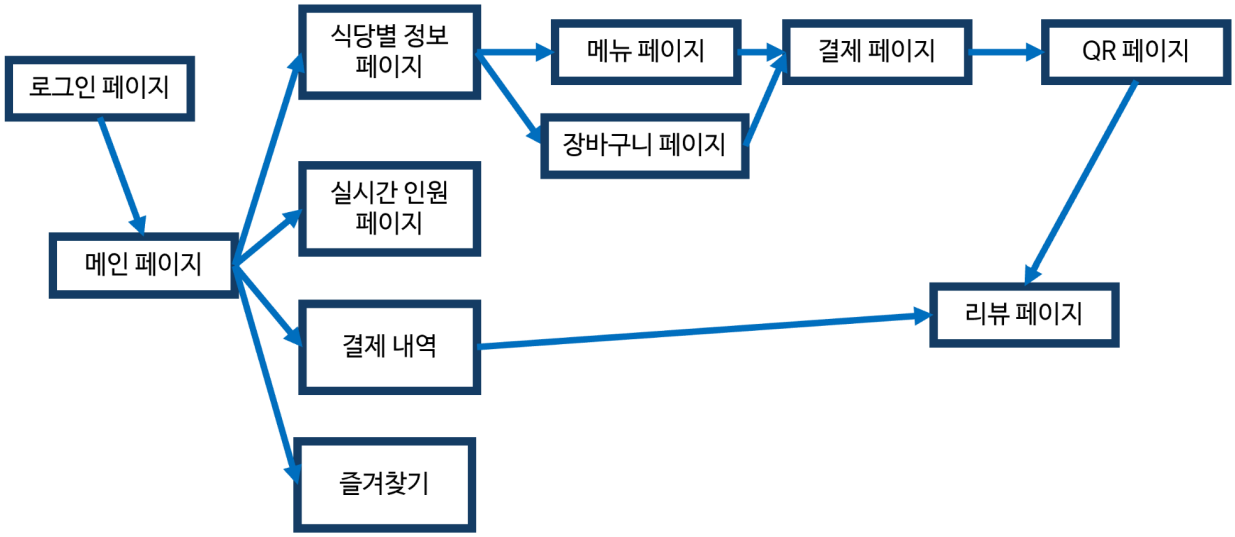
2.1.1 Market Status

성균관대학교의 학식 관련 정보를 얻을 수 있는 비슷한 서비스는 두 가지가 있다.

- **Everytime**
  에브리타임(Everytime)은 대학생을 위한 대표적인 커뮤니티 애플리케이션으로, 수업 시간표 관리, 강의 평가, 학식 정보 등의 학사 관련 기능을 제공한다. 성균관대학교 학생들 또한 이를 통해 학식 메뉴나 운영 정보를 확인할 수 있지만, 학식 관련 데이터는 최신 정보가 반영되지 않는 경우가 빈번하게 존재한다. 또한, 메뉴 리뷰 시스템이 존재하긴 하지만 사진 업로드 기능이 없어 시각적 정보 전달이 제한적이며, 메뉴의 품질이나 혼잡도에 대한 정량적인 정보가 부족하다. 결과적으로 에브리타임은 학식에 대한 단편적인 후기나 단시간의 정보 공유에는 유용하지만, 실시간 재고 관리·혼잡도 파악·결제 기능과 같은 실질적인 식당 운영 지원 기능은 제공하지 않는다.

- **SKKU CAMPUS MAP**(https://www.skku.edu/skku/about/campusInfo/campusMap.do)
  SKKU CAMPUS MAP은 성균관대학교 공식 홈페이지 skku.edu에서 제공하는 공식 캠퍼스 지도 서비스로, 학교 건물들의 위치를 한눈에 확인할 수 있는 웹 기반 애플리케이션이다. 각 건물의 명칭, 위치, 교내 시설 위치 등을 시각적으로 제공함으로써 방문객이 캠퍼스를 쉽게 탐색할 수 있도록 돕는다. 그러나 이 서비스는 단순한 건물 단위의 위치 정보 제공에 그치며, 식당 혼잡도나 학식 재고 현황, 메뉴 정보 등 실시간 학식 관련 데이터는 제공하지 않는다. 따라서 사용자는 실제 식당을 방문하기 전까지는 재고 부족이나 대기 상황을 미리 알 수 없으며, 학내 식당 이용의 효율성을 높이기에는 한계가 있다.

2.1.2 Overall Structure



[그림 1] Overall Structure

2.2 Product Functions

2.2.1 로그인 및 사용자 관리

프로그램을 처음 실행하면 가장 먼저 로그인 페이지가 나타난다. 사용자는 킹고 포털 로그인 방식을 통해 본인 인증을 진행하며, 로그인 완료 시 개인화된 정보가 반영된 메인 페이지로 이동한다. 로그인은 식당 주문 내역, 즐겨찾기, 리뷰 작성 등 개인 데이터를 지속적으로 관리하기 위해 필수적이다. 회원가입이 되어 있지 않은 사용자는 기본 정보를 입력하고 계정을 생성한 후 서비스를 이용할 수 있다. 로그인 정보는 보안 강화를 위해 암호화되어 저장된다.

2.2.2 메인 페이지 및 식당 정보 탐색

로그인 이후 사용자는 메인 페이지로 진입하며, 이곳에서 캠퍼스 내 식당 전체 목록을 확인할 수 있다. 메인 화면에서는 식당별 현재 운영 상태, 실시간 혼잡도, 재고 상태가 표시된다. 또한 사용자는 원하는 식당을 선택하여 상세 메뉴 페이지로 이동할 수 있으며, 최근 결제 내역과 즐겨찾기 목록도 함께 표시된다. 이를 통해 사용자는 식당 정보에 손쉽게 접근하고, 대기 시간을 고려하여 효율적으로 식당을 선택할 수 있다.

2.2.3 메뉴 선택 및 장바구니

식당을 선택하면 해당 식당의 메뉴 페이지가 나타나며, 사용자는 각 메뉴의 이름, 가격, 사진, 리뷰 평점을 확인할 수 있다. 원하는 메뉴를 선택하면 수량 조정과 함께 장바구니에 추가할 수 있으며, 실시간으로 총 결제 금액이 계산되어 표시된다. 장바구니 내에서는 메뉴를 삭제하거나 수량을 변경할 수 있고, 품절된 메뉴의 경우 자동으로 비활성화되어 주문이 제한된다. 이 기능은 사용자가 한 번의 결제 과정에서 여러 메뉴를 효율적으로 선택 및 관리할 수 있도록 돕는다.

2.2.4 결제 및 QR 코드 발급

결제 단계에서는 사용자가 선택한 메뉴를 확인한 후, 결제 수단(카카오페이, 네이버페이, 카드 등)을 선택할 수 있다. 결제 버튼을 누르면 주문이 확정되고, 시스템은 자동으로 결제 확인 화면과 함께 고유한 QR 코드를 생성한다. 사용자는 해당 QR 코드를 이용해 식당 수령 창구에서 주문 내역을 확인받고 음식을 수령할 수 있다. QR 코드는 일정 시간 후 자동 만료되며, 결제 내역은 사용자의 계정과 연동되어 결제 내역 페이지에서 확인할 수 있다. 이 기능을 통해 사용자는 대기 시간을 최소화하고, 직원은 빠른 주문 확인 및 처리 과정을 수행할 수 있다.

2.2.5 리뷰 및 피드백

결제가 완료된 사용자는 결제 내역 페이지에서 리뷰 작성을 진행할 수 있다. 사용자는 텍스트와 함께 사진을 첨부하여 음식의 품질, 양, 맛, 서비스 등에 대한 평가를 남길 수 있다. 리뷰는 다른 사용자에게 메뉴 선택 시 참고 자료로 제공되며, 평점이 누적되어 식당의 전반적인 만족도를 시각적으로 나타낸다. 또한 리뷰 데이터는 관리자 측에서 메뉴 개선 및 재고 관리 정책을 수립하는 데 활용된다. 이를 통해 사용자 경험을 개선하고, 식당 운영의 효율성을 높이는 선순환 구조를 형성한다.

2.2.6 즐겨찾기 및 최근 주문 관리

사용자는 자주 구매하는 메뉴를 즐겨찾기 목록에 등록할 수 있다. 즐겨찾기에 등록된 항목은 메인 페이지 상단에 고정되어, 반복 주문 시 빠르게 접근할 수 있다. 또한 최근 결제한 메뉴는 '최근 주문' 섹션에 자동으로 저장되어, 사용자는 최근 주문한 메뉴를 확인할 수 있다. 이 기능은 사용자의 편의성과 주문 효율성을 높이며, 시간 절약과 만족도 향상에 이바지한다.

2.3 User Classes and Characteristics

2.3.1 User

이 애플리케이션의 주요 사용자는 성균관대학교의 학생, 교직원으로 한정된다. 사용자는 기본적인 스마트폰 사용 능력을 보유하고 있으며, 애플리케이션을 통해 식당 정보를 탐색하고 메뉴를 선택 및 결제할 수 있는 수준의 디지털 활용 능력을 갖추고 있다고 가정한다. 또한 사용자는 한국어를 기본 언어로 사용하며, 앱 내 제공되는 직관적인 UI/UX 디자인을 통해 별도의 사용 설명 없이도 기능을 쉽게 이해하고 사용할 수 있다. 일반 학생 사용자의 경우 학내 식당의 혼잡도나 재고 상황을 빠르게 파악하고 효율적으로 식사 시간을 조정하는 데 목적이 있으며, 교직원은 주로 예약 주문 및 빠른 결제 기능을 활용한다.

2.3.2 System Administrator

본 애플리케이션의 시스템 관리자(System Administrator)는 서버 및 데이터베이스 운영, 사용자 계정 관리, 메뉴 및 재고 정보 업데이트 등을 담당하는 전문 인력으로 구성된다. 관리자는 시스템 전체 구조와 데이터 흐름을 충분히 이해하고 있으며, 오류나 예외 상황이 발생했을 때 이를 신속하게 감지하고 복구할 수 있는 능력을 갖추고 있다고 가정한다. 또한 관리자 계정은 일반 사용자보다 높은 접근 권한을 가지며, 메뉴 등록 및 수정, 식당 운영시간 변경, 재고 현황 갱신 등의 기능을 수행할 수 있다. 이들은 컴퓨터공학 또는 정보시스템 관련 지식을 바탕으로 데이터의 무결성과 보안을 유지해야 하며, 시스템 유지보수 시 사용자 개인정보 보호를 최우선으로 고려해야 한다. 마지막으로, 모든 관리자는 소프트웨어 윤리와 데이터 관리 지침을 숙지하고 이를 엄격히 준수해야 한다.

2.4 Operating Environment

2.4.1 Hardware

- Client: 사용자는 운영체제나 기기 종류에 상관없이 서비스를 이용할 수 있다. Flutter 기반으로 개발되어 Android, iOS의 환경에서 동일한 사용자 경험을 제공한다. 애플리케이션 구동을 위해서는 권장 2GB 이상의 메모리와 100MB 이상의 저장 공간이 필요하며, 최신 버전의 모바일 앱 실행 환경이 요구된다.

- Server: 서버는 Kubernetes 기반의 클러스터 환경에서 관리된다. 각 Pod는 Spring Boot 애플리케이션과 Nginx Reverse Proxy를 포함하며, 개별 서버의 최소 사양은 다음과 같다.
    - 1 vCPU 이상

- 512 MB 이상의 RAM
- SSD 기반 저장소 1GB 이상 (데이터 및 캐시 저장용)

모든 서버 인스턴스는 Kubernetes Node로 구성되어 자동 스케일링이 가능하며, 트래픽 부하에 따라 Pod가 동적으로 할당된다.

2.4.2 Software

- Client: 클라이언트는 Flutter 프레임워크를 사용하여 개발되었으며, 단일 코드 베이스로 다양한 플랫폼(Android, iOS)에 대응한다. UI 설계는 Figma를 기반으로 제작되었고, Flutter 위젯 구조에 최적화되어 있다. 클라이언트 애플리케이션은 Nginx를 통해 백엔드 API와 통신하며, HTTPS 프로토콜을 이용해 안전하게 데이터가 송수신된다.

- Server: 서버는 Spring Boot 프레임워크를 기반으로 개발되었으며, Nginx Reverse Proxy를 통해 클라이언트 요청을 처리한다. 애플리케이션은 Kubernetes 클러스터 내부에서 운영되며, 데이터베이스는 MySQL, 캐시는 Redis, 이미지 저장은 MinIO를 사용한다. 서버 환경은 Linux 기반이며, 컨테이너 관리 및 배포는 Kubernetes Cluster를 통해 자동화되어 있다. 백엔드 주요 구성 요소는 다음과 같다.
  - Spring Boot: API 서버 및 비즈니스 로직 처리
  - MySQL: 사용자 정보 및 주문 내역 데이터 관리
  - Redis: 세션 및 데이터 캐싱
  - MinIO: 이미지 및 파일 스토리지 관리
  - Nginx: 정적 리소스 제공 및 로드 밸런싱
  - Kubernetes: 서비스 배포, 스케일링 및 모니터링 자동화

2.5 Design and Implementation Constraints

시스템 설계 및 구현 방향에 대한 세부 사항은 다음과 같다.

1. 프론트엔드는 Flutter를 기반으로 구현되며, 단일 코드베이스로 Android, iOS 환경에서 동일하게 동작해야 한다.
2. 백엔드는 Spring Boot (v3.x) 프레임워크를 사용하며, Java 17 이상 버전에서 구동되어야 한다.
3. 모든 서버 애플리케이션은 Kubernetes Cluster 환경에서 Pod 단위로 배포되어야 하며, 자동 스케일링이 가능해야 한다.
4. 서버는 Nginx Reverse Proxy를 통해 클라이언트 요청을 처리하며, 로드 밸런싱 기능을 반드시 포함해야 한다.

5. 데이터베이스는 MySQL 8.0 이상을 사용하고, 캐시 서버는 Redis 7.0 이상 버전을 사용해야 한다.
6. 이미지 및 파일 저장소는 MinIO 객체 스토리지를 활용하며, 외부 네트워크 접근이 제한된 내부 환경에서 운영되어야 한다.
7. 서버 및 컨테이너는 최소 1 vCPU, 512 MB RAM, 그리고 1GB 이상의 SSD 저장소를 확보해야 한다.
8. 모든 서버 인스턴스는 Linux 기반 운영체제 위에서 동작해야 한다.
9. 클라이언트와 서버 간 통신은 반드시 HTTPS 프로토콜을 통해 이루어져야 하며, 모든 데이터 전송은 TLS 암호화를 거쳐야 한다.
10. 프론트엔드와 백엔드는 각각 독립적으로 배포 및 운영되며, 서로의 환경 변화에 영향을 받지 않도록 컨테이너 단위로 분리되어야 한다.
11. 시스템 내 모든 구성 요소는 Docker 기반 컨테이너로 관리되어야 하며, 버전 관리 및 롤백이 용이해야 한다.
12. 운영 및 유지보수를 위해 관리자는 Kubernetes 대시보드 또는 CLI를 이용해 Pod 상태와 리소스 사용량을 모니터링할 수 있어야 한다.

2.6 User Documentation

1. 애플리케이션의 설치 및 실행 방법, 로그인 절차, 결제 절차 등의 최소 요구사항을 명시해야 한다.
2. 지원되는 운영체제(Android, iOS) 및 최소 하드웨어 사양(메모리 128MB, 저장 공간 64MB 이상)을 문서에 포함해야 한다.
3. 사용자 개인정보(이름, 학번, 결제정보 등)의 수집, 저장, 이용 목적 및 정책을 명확히 기술해야 한다.
4. 모든 사용자 데이터는 HTTPS 및 TLS 암호화를 통해 안전하게 전송 및 저장된다는 점을 명시해야 한다.
5. 각 주요 기능(식당 정보 조회, 메뉴 선택, 결제, 리뷰 작성 등)에 대한 스크린샷과 사용 예시를 포함한 사용자 매뉴얼을 제공해야 한다.
6. 사용자 피드백이나 오류 보고를 위한 문의 기능 또는 개발자 연락 채널(예: 이메일, 문의 게시판)을 명시해야 한다.
7. 업데이트 내역 및 버전 변경 사항은 changelog 형식으로 문서화하여 지속적으로 관리해야 한다.

2.7 Assumptions and Dependencies

본 애플리케이션은 다음과 같은 가정 및 의존성을 기반으로 한다.

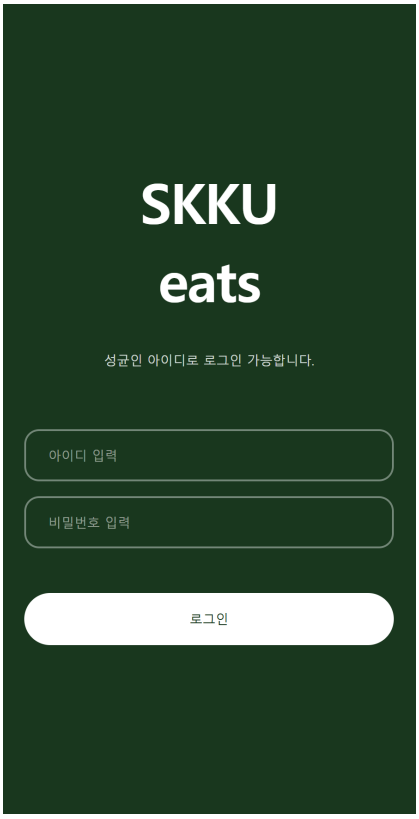
- 테스트 운영체제 환경: Android 14, iOS 17에서 테스트되었다.
- 서버 배포 환경: 성균관대학교에 구축된 Rocky Linux 9 기반의 Kubernetes Cluster로 운영된다.
- 의존 서비스: 데이터 저장은 MySQL 8.0, 캐싱은 Redis 7.0, 이미지 저장은 MinIO, 웹서버는 Nginx, 백엔드는 Spring Boot 3.x, 프론트엔드는 Flutter 3.x 버전을 기반으로 한다.
- 네트워크 환경: 모든 서비스는 HTTPS 기반 API 통신으로 연결되며, 서버와 클라이언트 간의 연결은 TLS 1.2 이상을 요구한다.

3. Specific Requirements

3.1 External Interface Requirements

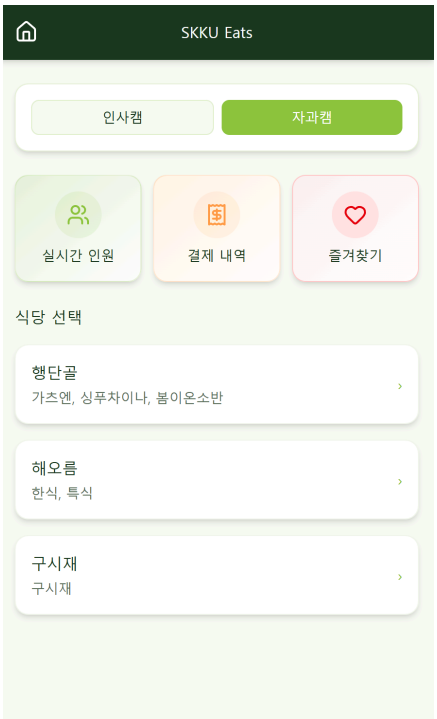
3.1.1 User Interface

Name	로그인 페이지
Purpose / Description	앱 시작 시 사용자가 성균인 아이디와 비밀번호를 입력하여 로그인하는 화면입니다.
Input source / Output destination	User Input / API Server
Time / Velocity	로그인 요청 후 1초 이내에 메인 페이지로 전환되어야 합니다.
Relationship with other inputs and outputs	로그인 성공 시 '메인 페이지'로 전환됩니다.

GUI Sketch	
Format and configuration of screen	1. 'SKKU eats' 로고와 "성균인 아이디로 로그인 가능합니다." 안내 문구가 표시됩니다. 2. 사용자는 '아이디 입력' 필드와 '비밀번호 입력' 필드에 값을 입력합니다. 3. '로그인' 버튼 클릭 시 시스템은 입력된 정보로 API 서버에 인증을 요청합니다. 4. 인증에 성공하면 '메인 페이지'로 이동합니다.
Data type	Text (ID), Text (Password)
Instruction type	API Call (Authentication), Screen Transition

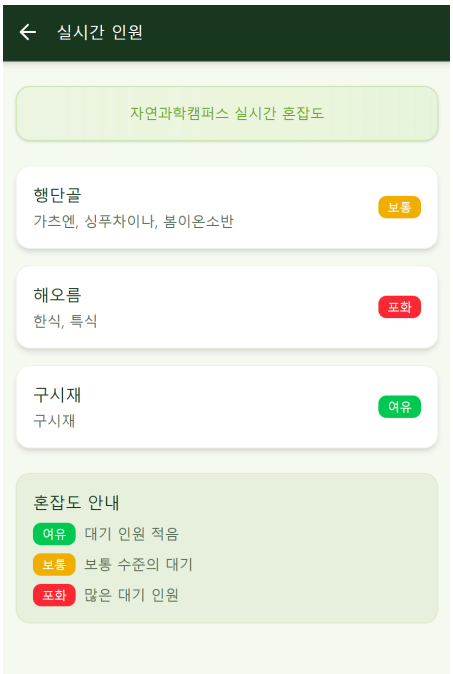
[표 3] Interface of Login page

Name	메인 페이지
Purpose / Description	로그인 후 보게 되는 앱의 메인 화면이다. 캠퍼스 선택, 식당 목록 확인 및 주요 기능(실시간 인원, 결제 내역, 즐겨찾기)으로 이동하는 허브 역할을 한다.

Input source / Output destination	User Input / App Navigation
Time / Velocity	각 버튼 클릭 시 1초 이내에 해당 페이지로 전환된다.
Relationship with other inputs and outputs	'로그인 페이지'에서 진입한다. '실시간 인원', '결제 내역', '즐겨찾기', '식당별 정보' 페이지로 이동할 수 있다.
GUI Sketch	
Format and configuration of screen	1. 화면 상단에 '인사캠', '자과캠'을 선택할 수 있는 토글 버튼이 있다. 2. 중앙에 '실시간 인원', '결제 내역', '즐겨찾기' 페이지로 이동하는 3개의 아이콘 버튼이 있다. 3. '식당 선택' 섹션에 현재 토글된 캠퍼스에 해당하는 학생 식당(행단골, 해오름 등) 목록이 리스트 형태로 표시된다. 4. 특정 식당 항목 클릭 시 해당 '식당별 정보 페이지'로 이동한다.
Data type	User Clicks (Buttons, Toggles)
Instruction type	Screen Transition, State Change (Toggle)

[표 4] Interface of Main page

Name	실시간 인원 페이지

Purpose / Description	캠퍼스 내 식당들의 실시간 혼잡도(대기 인원)를 '여유', '보통', '포화' 3단계로 시각적으로 제공한다.
Input source / Output destination	API Server / User Display
Time / Velocity	페이지 진입 시 1초 이내에 실시간 혼잡도 데이터를 로드한다.
Relationship with other inputs and outputs	'메인 페이지'에서 '실시간 인원' 버튼을 통해 진입한다.
GUI Sketch	
Format and configuration of screen	1. 상단에 현재 캠퍼스(예: 자연과학캠퍼스)의 실시간 혼잡도임이 표시된다. 2. 각 식당(행단골, 해오름 등)이 리스트 형태로 나열되며, 우측에 '여유'(초록), '보통'(주황), '포화'(빨강) 상태 태그가 표시된다. 3. 하단에 각 상태 태그의 의미를 설명하는 '혼잡도 안내' 섹션이 있다.
Data type	Real-time Data (String, Enum)
Instruction type	Data Fetch (API Call)

[표 5] Interface of Real-time Crowding page

Name	결제 내역 페이지

Purpose / Description	사용자의 과거 결제 내역을 시간순으로 보여주고, 주문 상태(미수령, 수령완료, 리뷰완료)에 따라 적절한 다음 행동(QR 확인, 리뷰 작성, 리뷰 확인)을 유도한다.
Input source / Output destination	API Server (History) / User Display
Time / Velocity	1초 이내에 결제 내역 리스트를 로드한다.
Relationship with other inputs and outputs	'메인 페이지'에서 '결제 내역' 버튼을 통해 진입한다. 각 버튼 클릭 시 'QR 페이지' 또는 '리뷰 작성 페이지' 등으로 이동할 수 있다.
GUI Sketch	
Format and configuration of screen	1. 결제 내역이 리스트 형태로 나열된다. 각 항목은 식당명, 메뉴 정보, 결제 시간, 가격, 주문 상태를 포함한다. 2. 주문 상태가 '미수령'인 경우: 'QR 확인하기' 버튼이 표시된다. 3. 주문 상태가 '수령완료'인 경우: '리뷰 작성하기' 버튼이 표시된다. 4. 주문 상태가 '리뷰완료'인 경우: '리뷰 확인하기' 버튼이 표시된다.
Data type	List (Payment Objects), User Clicks
Instruction type	Data Fetch, Screen Transition

[표 6] Interface of Payment History page

Name	즐겨찾기 페이지

Purpose / Description	사용자가 '식당별 정보 페이지'에서 '즐겨찾기 버튼(하트)'을 눌러 저장한 메뉴들만 모아서 보여주는 페이지이다.
Input source / Output destination	API Server (Favorites) / User Display
Time / Velocity	1초 이내에 즐겨찾기 목록을 로드한다.
Relationship with other inputs and outputs	'메인 페이지'에서 '즐겨찾기' 버튼을 통해 진입한다. 각 메뉴 항목 클릭 시 해당 '메뉴 페이지'로 이동할 수 있다.
GUI Sketch	
Format and configuration of screen	1. 화면 상단 네비게이션 바에 '즐겨찾기' 타이틀이 표시된다. 2. 사용자가 즐겨찾기한 메뉴들이 리스트 형태로 나열된다. 3. 각 항목은 메뉴명, 식당명/카테고리, 가격, 즐겨찾기 아이콘(하트), '상세보기' 버튼, '장바구니' 버튼을 포함한다. 4. '상세보기' 버튼 클릭 시 '메뉴 페이지'로, '장바구니' 버튼 클릭 시 장바구니에 아이템이 추가된다.
Data type	List (Favorite Menu Objects)
Instruction type	Data Fetch, Screen Transition

[표 7] Interface of Favorites page

Name	식당별 정보 페이지
Purpose / Description	특정 식당(예: 행단골)을 선택했을 때, 해당 식당에서 판매하는 모든 메뉴의 목록을 보여준다. 실시간 재고와 함께 장바구니/상세보기 기능을 제공한다.
Input source / Output destination	User Input / Cart, Menu Detail Page
Time / Velocity	1초 이내에 메뉴 목록 및 실시간 재고를 로드한다. '장바구니' 클릭 시 0.5초 이내에 피드백(예: 토스트 메시지)을 제공한다.
Relationship with other inputs and outputs	'메인 페이지'에서 식당 선택 시 진입한다. '상세보기' 클릭 시 '메뉴 페이지'로, '장바구니' 클릭 시 장바구니에 아이템이 추가된다.
GUI Sketch	
Format and configuration of screen	1. 상단에 식당명(예: 행단골)이 표시된다. 2. 하위에 메뉴 카테고리(전체, 가츠엔, 싯포차이나 등)를 선택하는 탭/필터가 있다. 3. 메뉴가 리스트 형태로 나열된다. 각 항목은 메뉴명, 가격, 실시간 재고('20개', '보통' 등), 즐겨찾기(하트) 버튼을 포함한다. 4. 각 메뉴마다 '장바구니' 버튼과 '상세보기' 버튼이 있다.
Data type	User Clicks, Real-time Data (Stock)

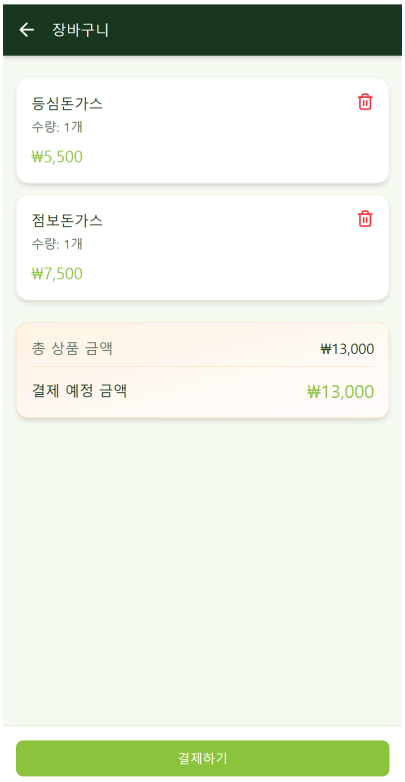
Instruction type	API Call (Add to Cart), Screen Transition

[표 8] Interface of Restaurant Menu page

Name	메뉴 페이지
Purpose / Description	개별 메뉴의 상세 정보(사진, 설명, 재고)와 해당 메뉴에 대한 사용자 리뷰 목록을 제공한다. 장바구니에 담거나 바로 결제할 수 있다.
Input source / Output destination	User Input / Cart, Payment Page
Time / Velocity	1초 이내에 메뉴 정보 및 리뷰 목록을 로드한다.
Relationship with other inputs and outputs	'식당별 정보 페이지'에서 '상세보기'를 클릭하여 진입한다. '장바구니' 클릭 시 아이템이 추가되고, '바로 결제' 클릭 시 '결제 페이지'로 이동한다.
GUI Sketch	
Format and configuration of screen	1. 상단에 메뉴 사진, 메뉴명, 가격, 상세 설명, 재고(예: 20개 남음)가 표시된다. 2. 하단에 '리뷰' 섹션이 있으며, 사용자들이 작성한 리뷰가 리스트 형태로 나열되어 스크롤로 확인할 수 있다.

	3. 화면 하단에 '장바구니' 버튼과 '바로 결제' 버튼이 고정(Fixed)되어 있다.
Data type	User Clicks, List (Review Objects)
Instruction type	API Call (Add to Cart), Screen Transition

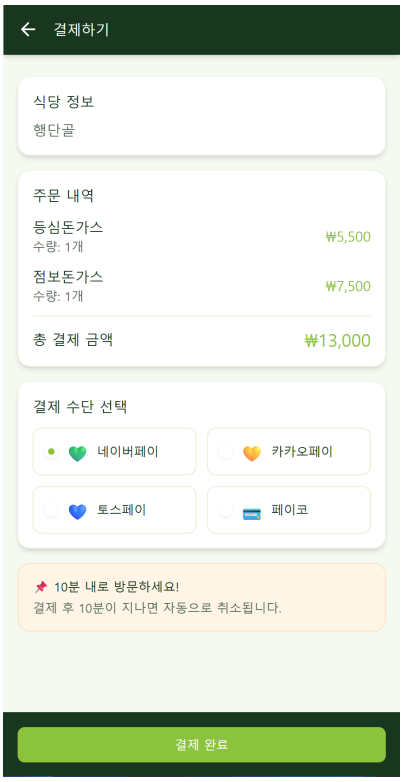
[표 9] Interface of Menu Detail page

Name	장바구니 페이지
Purpose / Description	사용자가 장바구니에 담은 항목들을 모아서 보여주고, 총 결제 금액을 확인 후 결제를 진행하도록 한다.
Input source / Output destination	User Input / Payment Page
Time / Velocity	'결제하기' 클릭 시 1초 이내 '결제 페이지'로 전환된다.
Relationship with other inputs and outputs	'식당별 정보' 또는 '메뉴' 페이지에서 진입한다. '결제하기' 클릭 시 '결제 페이지'로 이동한다.
GUI Sketch	

Format and configuration of screen	1. 상단에 장바구니에 담긴 메뉴 목록(수량, 개별 가격)이 표시된다. 각 항목마다 '삭제'(휴지통 아이콘) 버튼이 있다. 2. 중앙에 '총 상품 금액'과 '결제 예정 금액'이 표시된다. 3. 화면 하단에 '결제하기' 버튼이 고정되어 있다.
Data type	List (Cart Item Objects), User Clicks
Instruction type	Screen Transition, API Call (Update Cart)

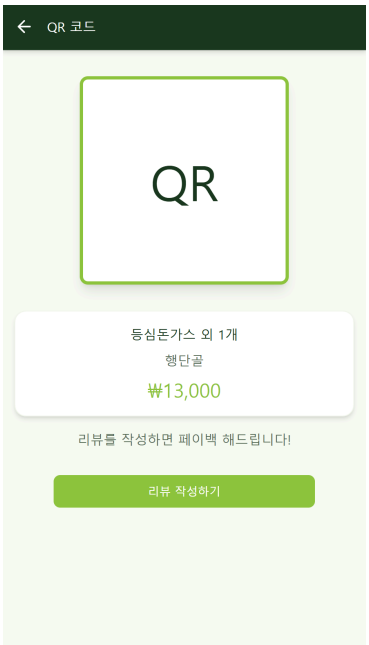
[표 10] Interface of Cart page

Name	결제 페이지
Purpose / Description	주문 내역을 최종 확인하고, 결제 수단을 선택하여 결제를 완료하는 페이지이다. (실제 결제는 생략한다.)
Input source / Output destination	User Input / QR Page
Time / Velocity	'결제 완료' 클릭 시 1초 이내 'QR 페이지'로 전환된다.
Relationship with other inputs and outputs	'메뉴 페이지'의 '바로 결제' 또는 '장바구니 페이지'의 '결제하기'를 통해 진입한다. '결제 완료' 클릭 시 'QR 페이지'로 이동한다.

GUI Sketch	
Format and configuration of screen	1. '식당 정보'와 '주문 내역' (메뉴 목록, 총 결제 금액)이 표시된다. 2. '결제 수단 선택' 영역에서 네이버페이 등 결제 옵션을 선택한다. 3. "10분 내로 방문하세요!"와 같은 결제 관련 중요 공지사항이 표시된다. 4. 화면 하단에 '결제 완료' 버튼이 있다.
Data type	User Clicks (Payment Method)
Instruction type	Screen Transition (Mock Payment)

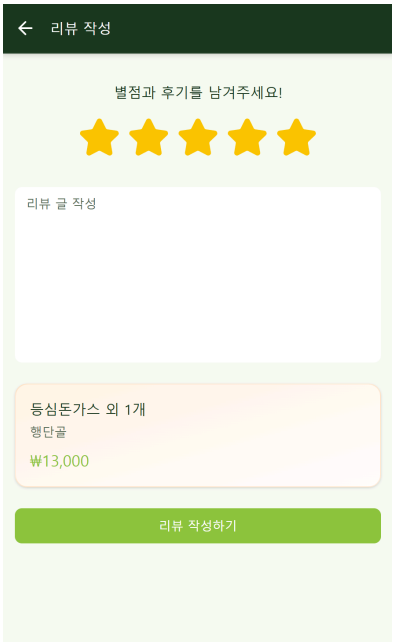
[표 11] Interface of Payment page

Name	QR 페이지
Purpose / Description	결제가 완료된 주문을 식당에서 수령할 때 인증하기 위한 QR코드를 표시한다. 또한, 식사 후 리뷰 작성을 유도한다.
Input source / Output destination	User Input / Review Writing Page
Time / Velocity	1초 이내에 QR 코드가 생성된다.

Relationship with other inputs and outputs	'결제 페이지'에서 '결제 완료' 시, 또는 '결제 내역'에서 '미수령' 상태의 'QR 확인하기' 클릭 시 진입한다. '리뷰 작성하기' 버튼 클릭 시 '리뷰 작성 페이지'로 이동한다.
GUI Sketch	
Format and configuration of screen	1. 중앙에 수령 인증을 위한 QR코드가 크게 표시된다. 2. 하단에 주문 요약(예: 등심돈가스 외 1개, 행단골, ₩13,000)이 표시된다. 3. "리뷰를 작성하면 페이백 해드립니다!"와 같은 리뷰 유도 문구가 있다. 4. '리뷰 작성하기' 버튼이 있다.
Data type	QR Code Image, User Click
Instruction type	Screen Transition, Data Fetch (QR)

[표 12] Interface of QR page

Name	리뷰 작성 페이지
Purpose / Description	사용자가 식사를 완료한 메뉴에 대해 별점과 텍스트 후기를 남길 수 있는 화면이다.
Input source / Output destination	User Input / API Server

Time / Velocity	'리뷰 작성하기' 버튼 클릭 시 1초 이내에 서버 전송 및 완료 피드백이 이루어져야 한다.
Relationship with other inputs and outputs	'QR 페이지' 또는 '결제 내역'의 '리뷰 작성하기' 버튼을 통해 진입한다.
GUI Sketch	
Format and configuration of screen	1. 상단에 "별점과 후기를 남겨주세요!" 문구와 5개의 별(Star Rating) 인터페이스가 있다. 2. 중앙에 '리뷰 글 작성'을 위한 텍스트 입력창(Text Area)이 있다. 3. 하단에 리뷰 대상 주문(예: 등심돈가스 외 1개)이 요약되어 표시된다. 4. '리뷰 작성하기' 버튼 클릭 시 입력된 별점과 텍스트가 서버로 전송된다.
Data type	User Input (Rating: 1-5, Text)
Instruction type	API Call (Submit Review)

[표 13] Interface of Review Writing page

3.1.2 Hardware Interfaces

Name	Applicable device for the system

Purpose / Description	본 애플리케이션은 Android 및 iOS 운영체제를 지원하는 모바일 기기에서 실행된다. 원활한 사용을 위해 최소 2GB 이상의 RAM과 100MB 이상의 여유 저장 공간을 권장한다.

[표 14] Hardware Interface

3.1.3 Software Interfaces

Name	Application Database
Purpose / Description	메뉴, 실시간 재고, 주문 내역, 사용자 정보 등 앱 운영에 필요한 모든 데이터를 관리하기 위한 쿼리 입력 및 출력을 위해 사용한다.
Input source / Output destination	Host Server / Database, Database / Host Server
Time / Velocity	데이터의 무결성을 보장하며 99.9% 이상의 가용성을 목표로 한다.
Relationship with other inputs and outputs	Query
GUI Sketch	Instant reaction
Format and configuration of screen	서버의 모든 데이터 관련 input/output과 연관된다.
Data type	None
Instruction type	None

[표 15] Software Interface

3.1.4 Communications Interfaces

Name	Client and Server
Purpose / Description	클라이언트(App)가 메뉴 조회, 재고 확인, 주문 요청(장바구니, 결제) 등을 서버에 전송하면, 서버는 해당 데이터를 반환하거나 처리 결과를 클라이언트로 전달한다.
Input source / Output destination	Client / Server, Server / Client

Time / Velocity	Packet
Relationship with other inputs and outputs	1초 이내의 API 응답을 목표로 한다. (단, 네트워크 환경에 따라 다를 수 있음)
GUI Sketch	클라이언트와 서버의 모든 input/output과 연관된다.
Format and configuration of screen	None
Data type	None
Instruction type	JSON, HTTPS

[표 16] Communication Interface

3.2 Functional Requirements

3.2.1 Use case

항목	내용
Use case name	로그인
Actor	사용자
Description	사용자가 성균인 ID를 사용하여 로그인한다.
Normal Course	1. 사용자가 로그인 페이지에 접속한다. 2. 성균인 학번(ID)과 비밀번호를 입력한다. 3. 입력한 정보가 서버의 회원 데이터와 일치하면 로그인에 성공한다. 4. 로그인 성공 시 메인 페이지로 이동한다.
Pre-Condition	사용자는 이미 성균인 ID를 보유한 상태여야 한다.
Post-Condition	로그인에 성공하면 메인 페이지 및 앱의 주요 기능에 접근할 수 있다.
Assumptions	서버와의 네트워크 연결이 정상이어야 한다.

[표 17] Use Case of Login and Authentication

항목	내용
Use case name	메인 페이지 진입
Actor	사용자
Description	사용자가 로그인 후, 오늘의 학식/식당 리스트, 실시간 인원 현황 등을 확인한다.
Normal Course	1. 로그인 완료 후 메인 페이지 로드. 2. 서버에서 학식 식당 리스트와 실시간 인원 정보를 불러온다. 3. 사용자는 원하는 식당을 선택한다.
Pre-Condition	사용자가 로그인 상태여야 한다.
Post-Condition	선택한 식당의 상세 페이지로 이동한다.
Assumptions	서버의 실시간 인원 API가 정상 작동 중이어야 한다.

[표 18] Use Case of Entering Main Page

항목	내용
Use case name	식당별 정보 확인
Actor	사용자
Description	사용자는 교내 식당의 영업시간, 메뉴, 현재 대기 인원 등의 정보를 확인할 수 있다.
Normal Course	1. 사용자가 메인 페이지에서 식당 목록을 선택한다. 2. 선택한 식당의 상세 페이지로 이동한다. 3. 앱은 해당 식당의 메뉴, 운영시간, 위치, 현재 인원 정보를 표시한다.

Pre-Condition	사용자가 로그인 상태여야 한다.
Post-Condition	사용자는 선택한 식당의 세부 정보를 확인할 수 있다.
Assumptions	식당 데이터는 관리자에 의해 사전에 등록되어 있어야 한다.

[표 19] Use Case of Viewing Restaurant Information

항목	내용
Use case name	메뉴 선택 및 결제
Actor	사용자
Description	사용자는 식당 메뉴를 선택하여 결제 과정을 진행한다.
Normal Course	1. 사용자가 식당 상세 페이지에서 메뉴를 선택한다. 2. 장바구니에 메뉴를 추가한다. 3. 결제 페이지로 이동하여 결제 수단을 선택한다. 4. 결제가 완료되면 주문 QR 코드가 생성된다.
Pre-Condition	사용자는 로그인 상태이고, 장바구니에 하나 이상의 메뉴가 존재해야 한다.
Post-Condition	결제가 완료되고, QR 코드가 화면에 표시된다.
Assumptions	결제 서버와의 연결이 원활해야 하며, 사용자의 결제수단이 유효해야 한다.

[표 20] Use Case of Menu Selection and Ordering

항목	내용
Use case name	장바구니 확인 및 수정

Actor	사용자
Description	사용자가 장바구니에 담긴 메뉴를 확인하고 수량 변경 또는 삭제 후 결제로 이동한다.
Normal Course	1. 사용자가 장바구니 페이지로 이동한다. 2. 담긴 메뉴 리스트와 총액이 표시된다. 3. 수량 조정 또는 삭제 후 '결제하기' 버튼을 누른다.
Pre-Condition	장바구니에 최소 1개 이상의 메뉴가 존재해야 한다.
Post-Condition	결제 페이지로 이동한다.
Assumptions	가격 정보는 서버 기준으로 자동 갱신된다.

[표 21] Use Case of Reviewing Cart

항목	내용
Use case name	결제
Actor	사용자, 결제 서버(PG)
Description	사용자가 장바구니 내 항목을 선택한 결제 수단으로 결제한다.
Normal Course	1. 사용자가 결제 페이지에서 수단(카드, 간편결제 등)을 선택한다. 2. 서버가 결제 요청을 PG 서버에 전달한다. 3. 결제 승인 후 QR 코드가 생성된다.
Pre-Condition	장바구니 정보가 최신이어야 하며, 사용자가 로그인 상태여야 한다.
Post-Condition	주문 내역이 생성되고 QR 페이지로 이동한다.
Assumptions	PG 서버와의 통신이 정상적으로 이뤄져야 한다.

[표 22] Use Case of Payment

항목	내용
Use case name	주문 QR 발급 및 제시
Actor	사용자, 식당 관리자
Description	결제 완료 후 발급된 QR 코드를 식당 직원에게 제시하여 주문을 확인받는다.
Normal Course	1. 결제 완료 시 QR 코드가 자동 생성된다. 2. 사용자는 식당에서 직원에게 QR을 제시한다. 3. 직원이 스캔 시 주문 상태가 “수령 완료”로 변경된다.
Pre-Condition	결제가 정상적으로 완료되어야 한다.
Post-Condition	주문 상태가 업데이트되고 리뷰 작성이 가능해진다.
Assumptions	QR의 유효시간 내에 제시해야 한다.

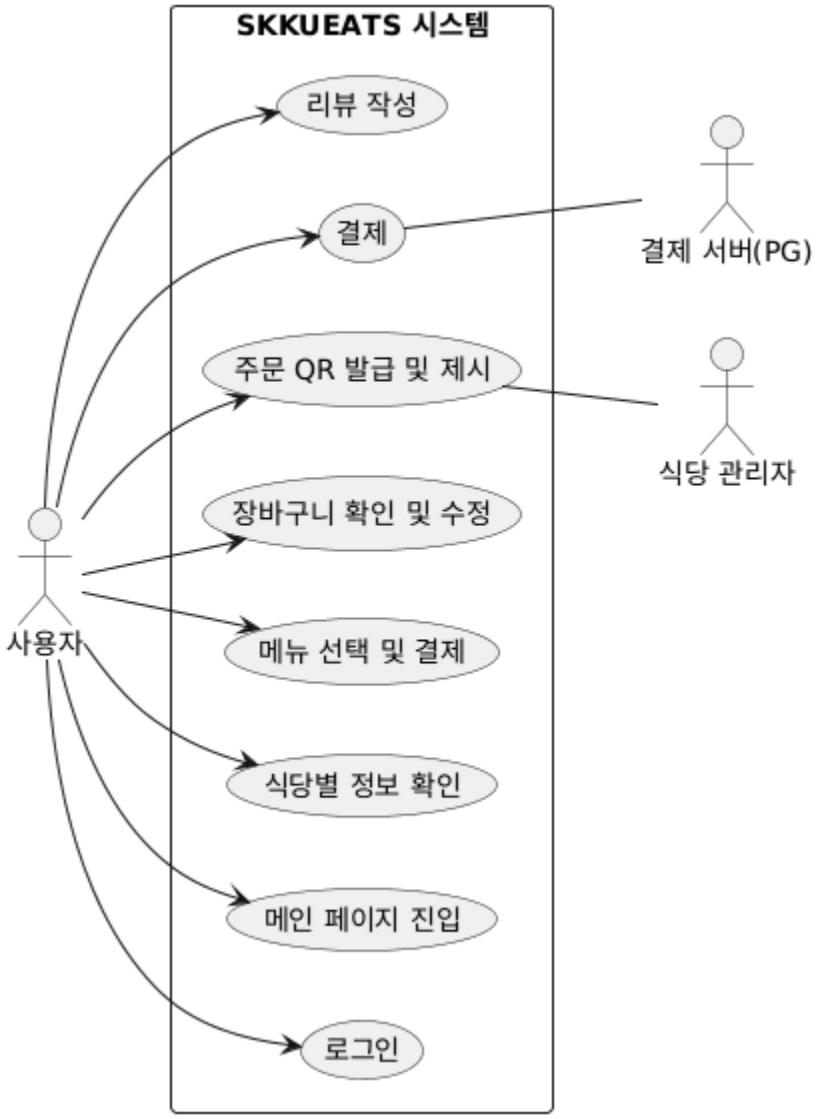
[표 23] Use Case of Checking QR

항목	내용
Use case name	리뷰 작성
Actor	사용자
Description	사용자는 식사 완료 후 결제 내역을 기반으로 리뷰를 작성할 수 있다.
Normal Course	1. 사용자가 결제 내역 또는 QR 페이지에서 리뷰 작성 버튼을 클릭한다. 2. 메뉴에 대한 평가와 코멘트를 입력한다. 3. 작성된 리뷰가 서버에 저장된다. 4. 리뷰는 다른 사용자에게 공개된다.

Pre-Condition	사용자는 결제를 완료한 상태여야 한다.
Post-Condition	리뷰가 성공적으로 등록되어 데이터베이스에 저장된다.
Assumptions	인터넷 연결이 정상이어야 하며, 서버가 응답 가능한 상태여야 한다.

[표 24] Use Case of Review Submission

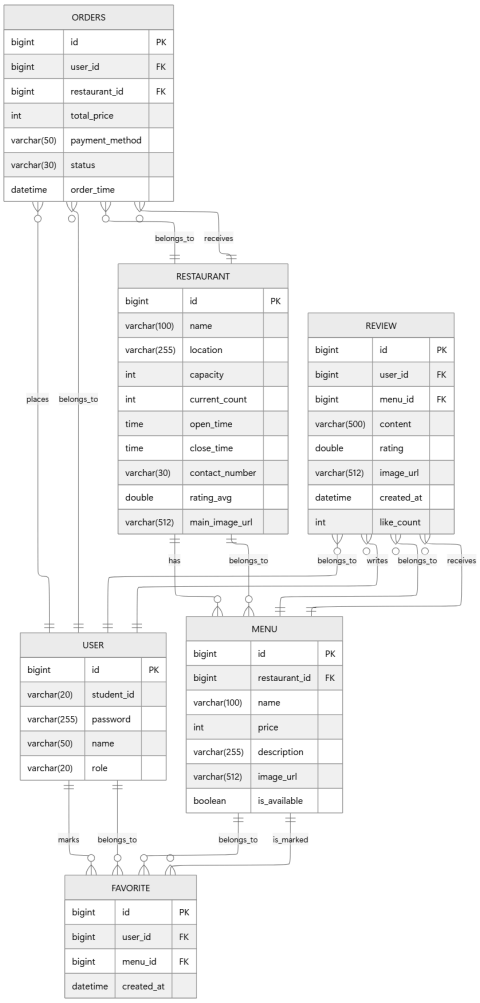
3.2.2 Use-Case Diagram



[그림 2] Use-Case Diagram

3.2.3 Data Dictionary (RDB)

RDB에는 사용자 정보와 주문 기록, 결제 데이터를 주로 저장한다.
아래는 ERD와 테이블 설명 표이다.



[그림 3] RDB ERD

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	사용자 고유 식별 번호
student_id	varchar(20)	Unique, NOT NULL	학번 기반 로그인 ID

password	varchar(255)	NOT NULL	암호화된 비밀번호
name	varchar(50)	NOT NULL	사용자 이름
role	varchar(20)	DEFAULT ‘user’	사용자 권한 (user/admin)

[표 25] User Data Dictionary

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	식당 고유 번호
name	varchar(100)	NOT NULL	식당 이름
location	varchar(255)	NOT NULL	식당 위치 정보
capacity	int		최대 수용 인원 수
current_count	int		실시간 인원 수
open_time	time		영업 시작 시간
close_time	time		영업 종료 시간
contact_number	varchar(30)		식당 연락처
rating_avg	double	DEFAULT 0	평균 평점
main_image_url	varchar(512)		대표 이미지 URL

[표 26] Restaurant Data Dictionary

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	메뉴 고유 번호
restaurant_id	bigint	FK → Restaurant.id	메뉴가 속한 식당 ID
name	varchar(100)	NOT NULL	메뉴 이름
price	int	NOT NULL	메뉴 가격
description	varchar(255)		메뉴 설명
image_url	varchar(512)		메뉴 이미지 URL
is_available	boolean	DEFAULT true	주문 가능 여부

[표 27] Menu Data Dictionary

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	주문 고유 번호
user_id	bigint	FK → User.id	주문자
restaurant_id	bigint	FK → Restaurant.id	주문한 식당
total_price	int	NOT NULL	주문 총 금액
payment_method	varchar(50)	NOT NULL	결제 수단 (카드, 간편결제 등)
status	varchar(30)	DEFAULT 'paid'	주문 상태 (paid, canceled, completed 등)

order_time	datetime	DEFAULT CURRENT_TIMESTAMP	주문 시각

[표 28] Order Data Dictionary

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	리뷰 고유 번호
user_id	bigint	FK → User.id	작성자
menu_id	bigint	FK → Menu.id	리뷰 대상 메뉴
content	varchar(500)	NOT NULL	리뷰 내용
rating	double	NOT NULL	평점 (1~5)
image_url	varchar(512)		리뷰 첨부 이미지
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	작성 시간
like_count	int	DEFAULT 0	좋아요 수

[표 29] Review Data Dictionary

Name	Type	Constraint	Description
id	bigint	PK, NOT NULL	즐겨찾기 고유 번호
user_id	bigint	FK → User.id	등록한 사용자
menu_id	bigint	FK → ―Menu.id	즐겨찾기 메뉴

created_at	datetime	DEFAULT CURRENT_TIMESTAMP	즐겨찾기 등록 시각

[표 30] Favorite Data Dictionary

3.3 Non-Functional Requirements

3.3.1 Product Requirements

3.3.1.1 Usability Requirements

1. 사용자는 **3**회 이하의 터치로 식당·메뉴 조회 및 주문 화면까지 도달할 수 있어야 한다.
2. 처음 사용하는 사용자는 **5**분 이내에 기본 기능(식당 선택, 메뉴 조회, 주문, 수령 방법)을 이해하고 사용할 수 있어야 한다.
3. 화면 내 주요 버튼(주문, 결제, 뒤로 가기 등)은 **44×44 pt** 이상 크기로 배치하여, 모바일 환경에서 쉽게 누를 수 있어야 한다.
4. 색상·아이콘·레이블은 일관된 스타일을 유지하며, 혼잡도(여유/보통/혼잡), 품절 여부, 특식 여부를 직관적인 아이콘과 색상으로 구분해 표시해야 한다.
5. 한국어를 기본 언어로 제공하며, 오류 메시지와 안내 문구는 일반 학생이 이해할 수 있는 수준의 자연어로 제공되어야 한다.

3.3.1.2 Efficiency Requirements

1. 홈 화면(캠퍼스별 식당 목록 및 혼잡도 요약)의 로딩 시간은 **2**초 이내여야 한다(정상적인 네트워크 환경 기준).
2. 특정 식당의 메뉴·재고·대기열 정보를 조회할 때 서버 응답 시간은 **2**초 이내여야 한다.
3. 대기 인원 및 재고 정보는 최소 **10**초 간격으로 갱신되며, 갱신 작업은 사용자의 화면 조작(스크롤, 탭 등)에 지장을 주지 않아야 한다.
4. 앱은 백그라운드 상태에서 불필요한 통신을 최소화하여, 하루 평균 데이터 사용량이 과도하게 증가하지 않도록 해야 한다.
5. 점심시간 피크(11:30~13:00) 기준 동시 접속 사용자 300명 이상 상태에서도 API 응답 시간은 2초 이내를 유지해야 한다.

3.3.1.3 Dependability Requirements

1. 주문·결제·재고 변경과 같은 중요 트랜잭션은 원자성(Atomicity)을 보장해야 하며, 장애 발생 시에도 데이터 불일치(돈은 결제되었으나 주문이 생성되지 않는 등)가 일어나지 않아야 한다.
2. 서버 장애 또는 네트워크 단절이 발생한 경우, 사용자는 적절한 안내 메시지를 받고, 재시도 버튼을 통해 동일 요청을 다시 시도할 수 있어야 한다.
3. 학기 중 평일 점심시간(11:00~14:00)을 기준으로 서비스 가용성은 **99%** 이상을 목표로 한다.
4. 노쇼 처리·재고 재오픈 로직은 자동화된 배치 또는 타이머에 의해 수행되며, 중복 실행으로 인해 재고가 잘못 증가/감소하지 않도록 해야 한다.
5. 장애·오류·예외 상황(결제 실패, 통신 오류, DB 오류 등)은 로그로 기록되어, 이후 분석 및 문제 해결이 가능해야 한다.

3.3.1.4 Security Requirements

1. 모든 클라이언트–서버 통신은 **HTTPS**를 사용하여 암호화해야 한다.
2. 비밀번호는 평문으로 저장하지 않으며, 일방향 해시 함수 **+** 솔트를 사용해 저장해야 한다.
3. 결제 관련 정보(카드 번호 등)는 서버에 직접 저장하지 않으며, PG사에서 제공하는 토큰·거래 ID만 저장한다.
4. 사용자는 본인 계정에 대해서만 주문 내역·즐겨찾기·리뷰를 조회/수정할 수 있어야 하며, 다른 사용자의 개인정보에 접근할 수 없어야 한다.
5. 관리자 페이지(식당 관리자, 시스템 관리자)는 역할 기반 권한 관리(RBAC)를 적용하여, 일반 사용자가 접근할 수 없도록 한다.
6. 로그인 실패가 일정 횟수(예: 5회)를 초과할 경우, 일정 시간 동안 로그인을 제한하는 등 계정 보호 정책을 적용한다.
7. 개인정보 조회/수정 로그는 최소 90일 이상 보관해야 한다.

3.3.2 Organization Requirements

3.3.2.1 Environmental Requirements

1. 모바일 클라이언트는 **Android 9.0/iOS 14** 이상에서 정상 동작하는 것을 기본 목표로 한다.
2. 서버 애플리케이션은 리눅스 기반 환경(예: Ubuntu LTS)에서 동작하도록 개발하며, 최소 사양은 CPU 1코어, 메모리 512 MB, 디스크 1GB 이상을 가정한다.

3. 시스템은 캠퍼스 **Wi-Fi, LTE/5G** 환경 모두에서 사용 가능해야 하며, 일시적 네트워크 불안정 상황에서도 최소한의 오류 처리 및 재시도 기능을 제공해야 한다.
4. 과제 수행 시 실제 학교 내부 시스템(GLS, POS 등)과 연동이 어려운 경우, 모의**(Mock)** 서버 또는 더미 데이터로 대체 가능하다.

3.3.2.2 Development Requirements

1. 전체 아키텍처
    a. 애플리케이션은 쿠버네티스**(Kubernetes)** 클러스터 상에서 동작하며,
        i. Flutter 클라이언트
        ii. Nginx
        iii. Spring 기반 백엔드
        iv. MySQL, MinIO, Redis로 구성된 마이크로 서비스 형태(또는 멀티 컴포넌트 구조)로 개발한다.
2. 클라이언트**(**모바일 앱**)**
    a. 모바일 애플리케이션은 **Flutter**로 개발한다.
    b. 서버와의 통신은 HTTPS 기반 REST API(JSON 포맷)를 사용한다.
    c. 빌드 타깃은 Android와 iOS를 기본으로 한다.
3. 백엔드 애플리케이션
    a. 백엔드는 **Spring Boot**를 사용하여 개발한다.
    b. 계층 구조(Controller–Service–Repository)를 유지하여 유지보수성을 확보한다.
    c. 인증/인가, 주문·재고 처리, 노쇼 처리, 추천 로직 등 핵심 비즈니스 로직을 담당한다.
4. 웹 서버 **/** 리버스 프록시
    a. 클라이언트의 모든 요청은 **Nginx**를 통해 수신되며,
        i. 정적 리소스 제공,
        ii. 백엔드 서비스로의 리버스 프록시,
        iii. 로드 밸런싱(필요 시)을 담당한다.
5. 데이터베이스 및 스토리지
    a. 관계형 데이터(사용자, 식당, 메뉴, 주문, 리뷰 등)는 **MySQL**에 저장한다.
    b. 세션 정보, 캐시(예: 실시간 대기 인원, 인기 메뉴 등)는 **Redis**를 사용해 관리한다.
    c. 포토 리뷰 이미지, 배너 이미지 등 대용량 파일은 **MinIO**(S3 호환 오브젝트 스토리지)에 저장하고, DB에는 경로/URL만 저장한다.
6. 컨테이너 및 배포

   a. Spring Boot 백엔드 애플리케이션은 **Docker** 컨테이너로 패키징한다.
   b. 컨테이너는 **Kubernetes Deployment/StatefulSet, Service, Ingress** 등을 이용해 배포·운영한다.
   c. 개발 환경에서는 Minikube 등 로컬 클러스터를 사용할 수 있으며, 운영 환경은 클라우드(Kubernetes 서비스)를 가정한다.
7. **CI/CD** 및 형상 관리
   a. 소스 코드는 Git 저장소로 관리하며, 브랜치 전략(main + feature 브랜치)을 적용한다.
   b. 가능하다면 GitHub Actions, GitLab CI 등으로 빌드·테스트·도커 이미지 빌드를 자동화한다.
   c. 주요 브랜치에 머지될 때마다 Kubernetes 클러스터에 자동 또는 반자동 배포하는 흐름을 목표로 한다(과제에서는 설계 수준으로 명시해도 무방).
8. 테스트 및 품질 관리
   a. 백엔드의 주요 비즈니스 로직에 대해 JUnit 기반 단위 테스트를 작성한다.
   b. API 동작은 Postman/Swagger 등을 사용해 수동 또는 자동 테스트한다.
   c. Flutter 측에서는 최소한 핵심 화면(로그인, 식당 조회, 주문 흐름)에 대해 기본 동작 테스트를 수행한다.
9. 로깅 및 모니터링
   a. 백엔드는 요청/응답, 오류, 주요 이벤트(주문 생성, 결제 결과 등)에 대한 로그를 남긴다.
   b. Kubernetes 환경에서 로그 수집/모니터링을 위해 kubectl logs 또는 간단한 로깅 스택(EFK, Loki 등)을 사용할 수 있도록 고려한다.

3.3.3 External Requirements

3.3.3.1 Regulatory Requirements

1. 사용자의 개인정보(이메일, 이름, 학번 등) 처리 시 개인정보보호법 및 정보통신망법 등 관련 법규를 준수해야 한다.
2. 결제 기능을 실제로 구현할 경우, 전자금융거래법 및 **PG**사의 약관을 준수해야 하며, 과제 수준에서는 실제 결제 대신 테스트 결제·모의 결제로 대체할 수 있다.
3. 사용되는 오픈소스 라이브러리(예: UI 라이브러리, 서버 프레임워크 등)는 각자의 라이선스(예: MIT, Apache 2.0)를 준수해야 하며, 라이선스 목록을 문서 부록에 명시한다.

4. 사진 리뷰 등 사용자가 업로드한 콘텐츠에 대해, 저작권·초상권을 침해하지 않도록 이용약관에 관련 내용을 명시한다.

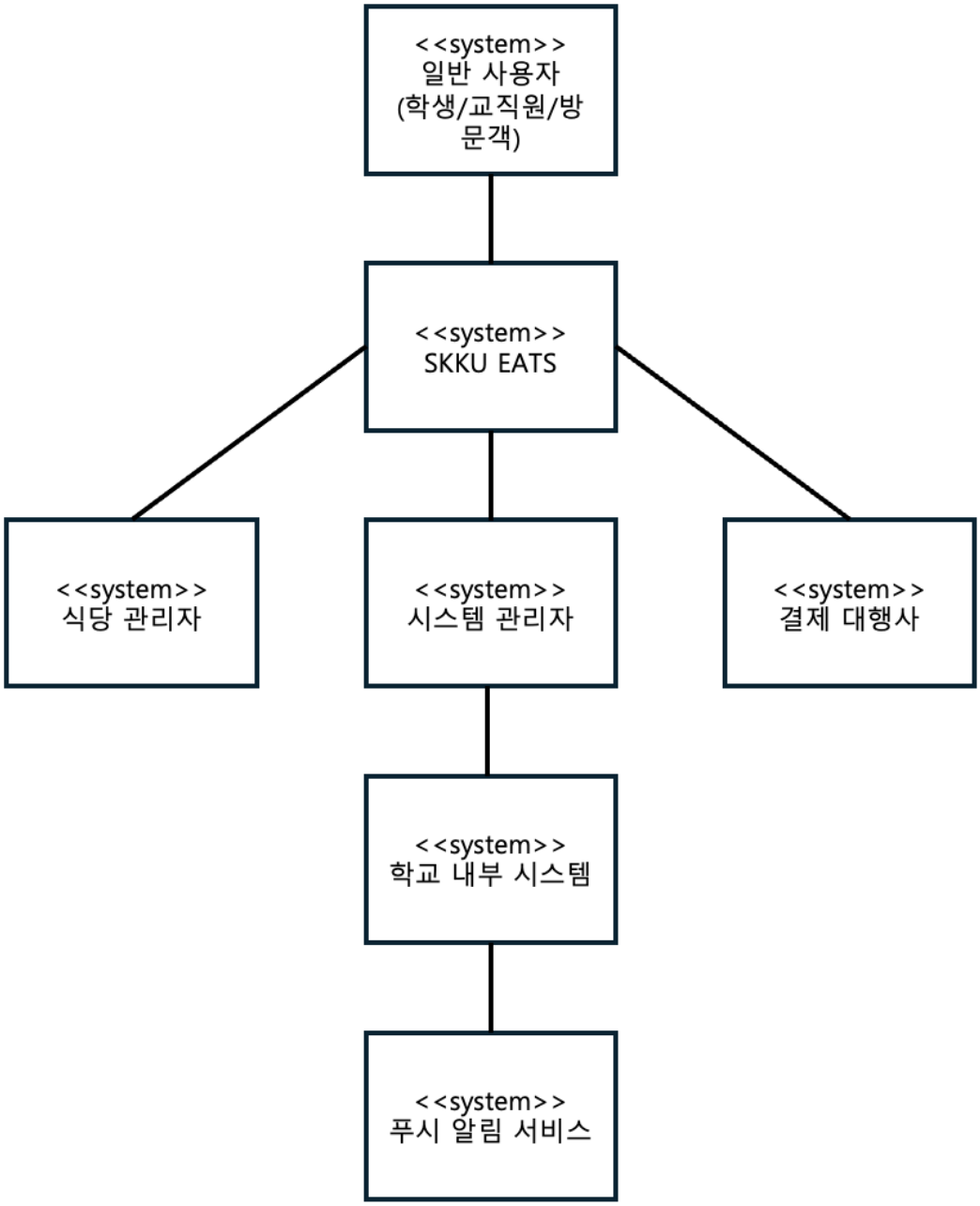
3.3.3.2 Ethical Requirements

1. 리뷰 기능은 특정 개인·집단에 대한 비방·차별·혐오 표현을 허용하지 않으며, 운영자는 부적절한 리뷰를 삭제 또는 비공개 처리할 수 있다.
2. 수집된 데이터(선호 메뉴, 방문 기록 등)는 서비스 품질 향상 및 통계 목적 이외의 용도로 사용하지 않는 것을 원칙으로 한다.
3. 사용자의 위치 정보, 알레르기 정보 등 민감한 정보는 최소한으로 수집하며, 사용자가 언제든지 수정·삭제할 수 있게 한다.
4. 알림(푸시 메시지)은 사용자의 동의 하에만 발송하며, 과도한 마케팅·광고성 알림은 지양한다.

3.3.4 Organizing the Specific Requirements

3.3.4.1 Context Model

본 시스템의 컨텍스트는 다음과 같이 정의한다.



[그림 4] System Context Model

3.4 API Specification

해당 문서에서는 본 프로젝트의 여러 API 중 주요한 API에 대한 명세를 제공한다.

1. 로그인 API

성균인 아이디와 비밀번호를 받아 성균인 계정인지 확인하고 엑세스 토큰과 리프레시 토큰을 발급해 반환한다. 만약 해당하는 계정이 없으면 오류 메시지를 반환한다.

[POST] /api/v1/auth	Example
Authentication	X
QueryString	X
Path Parameter	X
Body	id: string password: string

Response(200 OK)	{
	“access_token”: “eyJhbG….GoHh4”,
	“refresh_token”: “eyJhbG….4E4Ho”
	}

Response(400 Bad Request)	{ “code”: 20419, “message”: “Login Failed” }

[표 31] 로그인 API의 파라미터 및 응답 형식 예시

2. 혼잡도 조회 API

캠퍼스 별 식당의 혼잡도 여부를 조회한다. Path Parameter를 통해 캠퍼스를 입력하면, 해당 캠퍼스에 존재하는 식당에 대한 혼잡도 여부를 반환한다.

[GET] /api/v1/cafeteria/{campus}/status	Example
Authentication	X
QueryString	X
Path Parameter	campus: int
Body	X

Response(200 OK)	[{ “name”: “행단골”, “status”: 2 }, { “name”: “해오름”, “status”: 1 }]
Response(400 Bad Request)	{ “code”: 3193, “message”: “Invalid Campus Code” }

[표 32] 혼잡도 조회 API의 파라미터 및 응답 형식 예시

3. 식단 조회 API

캠퍼스 별, 식당별 식단 정보를 조회한다. 각 메뉴의 가격과 재고 등의 정보를 제공한다.

[GET] /api/v1/cafeteria/{campus}/{cafeteriaName}	Example
Authentication	X
QueryString	X
Path Parameter	campus: int cafeteriaName: string
Body	X
Response(200 OK)	[{ “id”: 3308, “name”: “등심돈까스”, “price”: 5500, “stock”: 20 }, { “id”: 2205, “name”: 점보돈까스”, “price”: 7500, “stock”: 10 }]
Response(400 Bad Request)	{ “code”: 3193, “message”: “Invalid Campus Code” }

[표 33] 식단 조회 API의 파라미터 및 응답 형식 예시

4. 리뷰 작성 API

해당하는 메뉴에 리뷰를 작성한다. 날짜는 데이터베이스에서 객체가 생성된 시각을 자동으로 매핑된다. 이름은 서비스 레이어에서 마스킹처리하여 데이터베이스에 저장한다. 사용자가 해당 메뉴를 구매한 이력이 없는 경우 오류를 반환한다.

[POST] /api/v1/menu/{id}/review	Example
Authentication	Bearer access_token

QueryString	X
Path Parameter	id: int
Body	point: float review: string
Response(200 OK)	[{ “point”: 4.0, “name”: “정**”, “review”: “등심돈까스 부드러워요”, “date”: “2025-11-01” }]
Response(400 Bad Request)	{ “code”: 3197, “message”: “Invalid Menu Id” }

[표 34] 리뷰 작성 API의 파라미터 및 응답 형식 예시

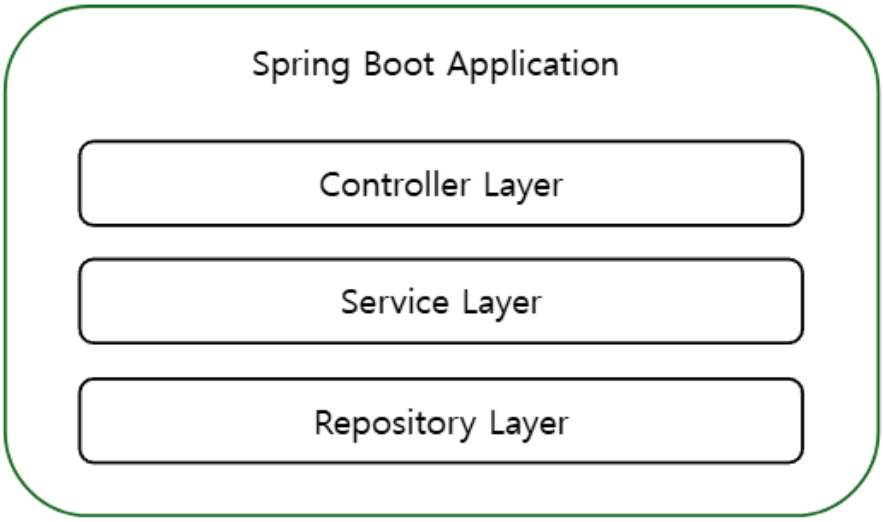
5. 결제 API

사용자가 장바구니에 담은 메뉴를 결제한다. 사용자가 선택한 간편결제 서비스를 통해 결제를 진행하고, 결제 결과를 서버에서 검증하여 최종 결제 여부를 판단하고 그 결과를 데이터베이스에 저장한다.

[POST] /api/v1/payment	Example
Authentication	Bearer access_token
QueryString	X
Path Parameter	X
Body	orderList: List<object> paymentMethod: int paymentStatus: int
Response(200 OK)	{ “status”: 1, “qrCode”: “” }
Response(400 Bad Request)	{ “code”: 3197, “message”: “Invalid Menu Id” }

[표 35] 결제 API의 파라미터 및 응답 형식 예시

3.5 Server Architecture



[그림 5] 서버 구조

서버는 Spring Boot 프레임워크를 기반으로 구축되며, 상태를 갖지 않는 Stateless를 지향한다. 서버의 전체 시스템은 Layered Architecture(계층형 구조)로 구성된다. 각 계층은 역할이 명확히 분리되어 있고 서로 직접적인 의존성을 갖지 않으며, DTO(Data Transfer Object)를 이용하여 필요한 데이터만을 전달함으로써 결합도를 낮춘다.

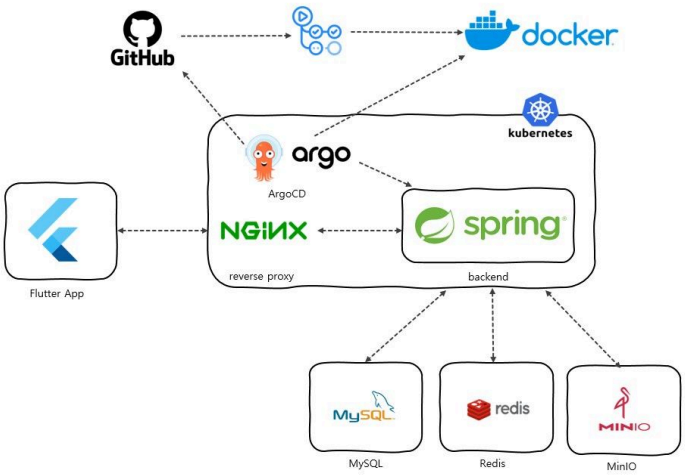
서버는 모바일 애플리케이션과 완전히 독립적으로 동작하며, 두 시스템 간의 데이터 교환은 오직 HTTP 기반 REST API를 통해 이루어진다. 이를 통해 클라이언트 플랫폼(Android, iOS 등)과 무관하게 동일한 API를 제공할 수 있으며, 서버는 클라이언트의 UI나 실행 환경에 영향을 받지 않는 독립적인 애플리케이션으로 유지된다.

- Controller Layer
  - 클라이언트로부터 HTTP 요청을 가장 먼저 전달받는 계층이다.
  - 요청 데이터를 DTO 형태로 변환한 후 Service Layer로 전달하며, 처리 결과를 다시 DTO로 변환하여 JSON 형태로 응답한다.
  - 예외 처리, 인증 및 인가, 파라미터 검증의 작업이 해당 레이어에서 수행된다.
- Service Layer
  - 핵심 비즈니스 로직이 위치하는 계층으로, Controller와 Repository 사이의 중간 역할을 수행한다.

    - 도메인 규칙에 따라 데이터를 가공하고 트랜잭션을 관리하며, 복수의 Repository를 조합하여 작업을 처리한다.
    - 데이터베이스 접근이 필요한 경우 요청에 필요한 정보만 담은 DTO를 Repository Layer에 전달한다.
- Repository Layer
    - 데이터베이스 또는 외부 저장소에 직접 접근하는 계층이다.
    - 서버에서는 여러 데이터 저장소를 사용하므로, Repository Layer는 이를 추상화하여 통합적으로 다룰 수 있도록 설계한다.
    - Spring Data JPA, Spring Data Redis 등을 기반으로 구현하며 데이터베이스에 데이터를 생성, 수정, 조회, 삭제 하는 기능을 담당한다.
        - MySQL: 일반적인 사용자 정보, 리뷰, 로그 등 관계형 데이터 저장
        - Redis: 세션 및 토큰 정보, 자주 조회되는 식단 데이터 캐싱
        - MinIO: 이미지 등 정적 리소스 저장
- DTO와 Entity 분리 설계
    - 도메인 내부에서 사용하는 Entity 클래스는 외부에 노출되지 않으며, 데이터 일관성과 도메인 규칙을 유지하기 위해 Service Layer에서만 사용된다.
    - 반면, 클라이언트와의 데이터 교환 시에는 항상 DTO(Data Transfer Object)를 사용하여 필수 데이터만 전달한다.
    - 이를 통해 보안성과 캡슐화를 강화하고, API 변경 시 Entity 구조에 영향을 받지 않도록 유연성을 확보한다.

3.6 Infrastructure Architecture

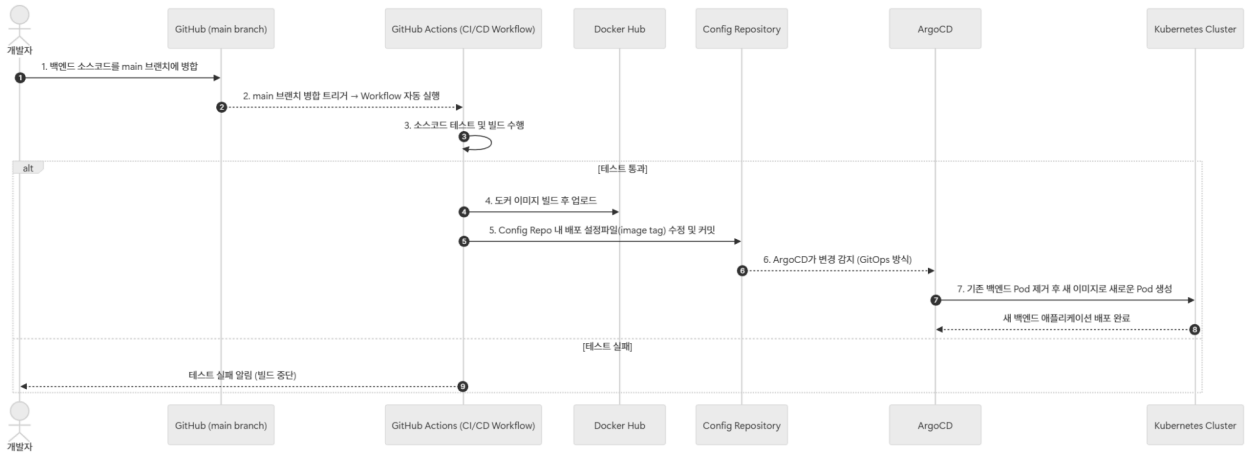
3.6.1 Overview



[그림 6] 인프라 아키텍쳐

인프라는 개발에 집중할 수 있도록 통합 및 배포 과정을 최대한 자동화한다. 이와 함께 부하가 많아지는 피크타임에는 오토 스케일링이 일어날 수 있도록 한다.

3.6.2 애플리케이션 배포



[그림 7] 애플리케이션 배포 과정에 대한 상세 시퀀스 다이어그램

쿠버네티스에 배포되는 백엔드 애플리케이션은 다음 시나리오를 따라 배포가 진행된다.

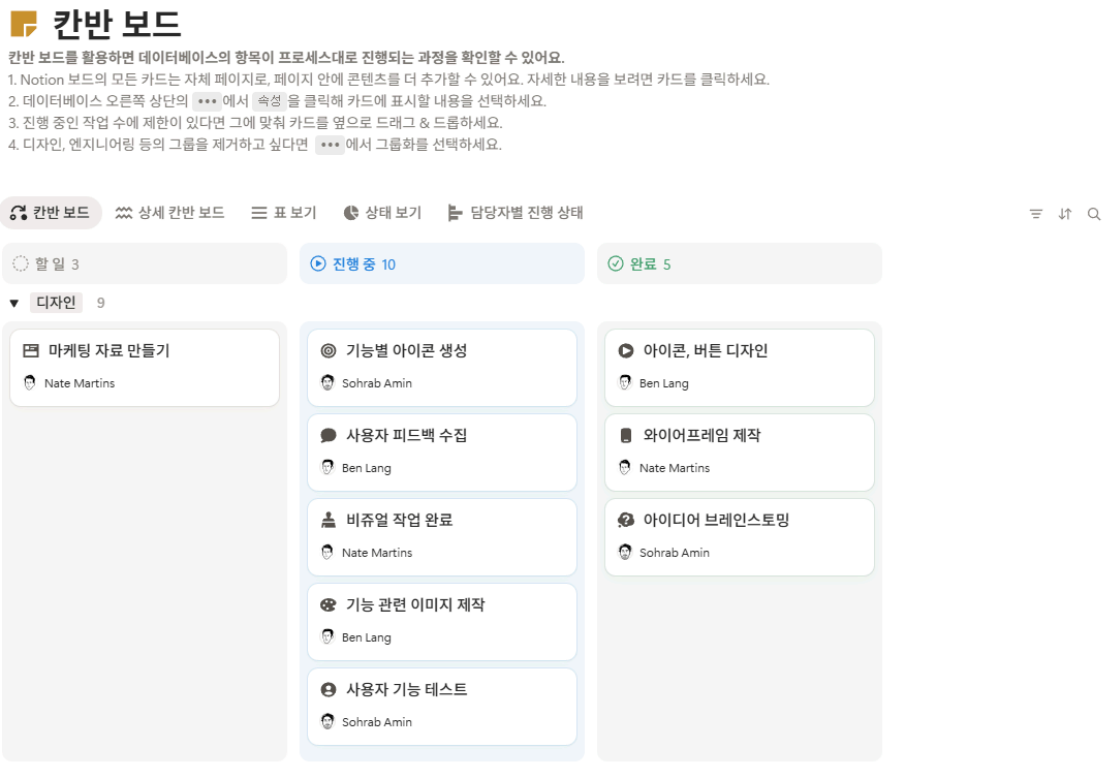
1. 개발자가 작성한 백엔드 소스코드는 main 브랜치에 병합된다.

2. main 브랜치에 병합할 때 Github Action에 있는 트리거를 통해 자동으로 워크플로우를 실행한다.
3. main 브랜치에 있는 소스코드를 테스트하고, 테스트가 통과되면 빌드하여 도커 이미지를 생성한다.
4. 빌드된 도커 이미지는 도커 허브에 업로드되고, 백엔드 배포 설정파일이 있는 Config 레포지토리에 이미지 정보를 넣어 수정한다.
5. ArgoCD가 Config 레포지토리의 수정 기록을 파악하고, 새로운 도커 이미지를 받아 기존 백엔드 Pod를 제거하고 새로운 백엔드 Pod를 만들어 쿠버네티스 클러스터에 배포한다.

3.6.3 인프라 운영

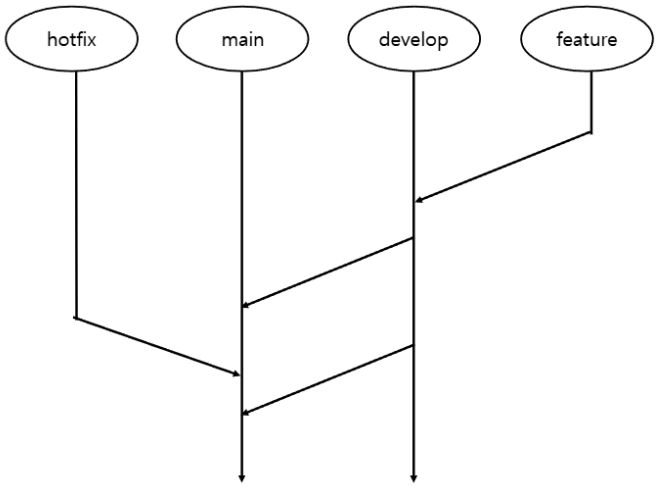
쿠버네티스 클러스터와 데이터베이스는 기존에 성균관대학교 내에 구축되어 있는 클러스터와 데이터베이스를 사용함에 따라 추가적인 인프라 구축 비용이 발생하지 않는다. 데이터를 성균관대학교 내에서 관리하여 개인정보 유출 등의 위험을 줄일 수 있다. 자주 사용되는 데이터를 캐싱하기 위해 Redis를 사용하고, 이미지 등 정적 파일을 저장하기 위해 MinIO를 사용한다. 트래픽이 증가하는 것을 대비하여 HPA(Horizontal Pod Autoscaler) 를 적용하여 부하가 많아지면 Pod의 개수를 자동으로 스케일링한다.

3.7 Project Management



[그림 8] 노션으로 구축한 칸반보드 예시

효율적인 프로젝트 진행을 위해 Notion 기반의 칸반보드를 구축하여 이용한다. 칸반보드는 업무의 진행 단계를 시각적으로 관리할 수 있는 애자일 도구로, 각 작업을 할 일(To do) - 진행 중(In progress) - 완료(Done) 단계로 시각화하여 업무 흐름을 파악할 수 있으며, 우선순위 조정과 병목 구간 확인에도 효과적이다. 디자인, 앱 개발, 백엔드 개발, 인프라, 총괄 담당이 동일한 칸반 보드를 공유함으로써 진행 상황을 실시간으로 공유하고 담당 업무간의 의존 관계를 빠르게 조율할 수 있다. 이와 함께 일주일에 한 번의 스크럼 미팅을 진행하여 모든 팀원이 전체 진행 상황을 파악하도록 한다.



[그림 9] 코드 관리 관련 브랜치

버전 관리 시스템은 Git을 기반으로 하며, 원격 저장소는 Github을 활용한다. 개발 과정에서는 간소화된 Git Flow 전략을 적용하여 코드 품질과 협업 효율을 동시에 확보한다. 간소화된 Git flow 전략에서는 main, develop, feature, hotfix 브랜치를 사용한다.

- 하나의 feature 브랜치마다, 하나의 개발 기능을 담당한다.
- feature 브랜치에서 기능 개발이 끝날 때 마다, develop 브랜치로 커밋한다.
- develop 브랜치에서 개발이 완료된 버전은 테스트를 진행하고 main 브랜치에 배포한다.
- main 브랜치(프로덕션 애플리케이션)에서 발생한 문제는 hotfix 브랜치를 통해 해결 후 반영한다.
- 병합 과정에서 충돌이 발생하지 않는 것이 이상적이지만, 충돌이 발생하면 마지막에 작업을 진행한 담당자가 충돌을 해결한다.

3.8 System Evolution

3.8.1 Limitation and Assumption

본 시스템은 학식 주문 및 결제 서비스를 중심으로 개발되었으며, 현재는 교내 식당의 메뉴 조회, 실시간 인원 확인, QR 결제 및 리뷰 작성 기능에 한정되어 있다. 그러나 식당 외의 편의시설(카페, 휴게실, 매점 등)에 대한 서비스는 제공되지 않아 사용자의 활용 범위가 제한될 수 있다. 또한, 초기 버전에서는 학식 재고 데이터 및 혼잡도 정보가 일부 식당에 한해 수집되므로, 전체 식당의 실시간 정보 정확도는 다소 낮을 가능성이 있다.

본 서비스는 성균관대학교 재학생을 주요 사용자로 설정하고 있으며, 외부인 또는 졸업생의 접근은 제한될 수 있다. 사용자층이 학생 위주로 형성되기 때문에, 서비스 확장 시 다양한 연령층 및 외부 방문객을 고려한 UI/UX 보완이 필요하다. 또한, 학식 결제 시스템은 교내 네트워크 환경에 의존하는 구조이기 때문에, 인터넷 연결이 불안정한 환경에서는 기능이 원활하게 작동하지 않을 수 있다.

3.8.2 Anticipated Changes

향후 시스템은 학식 정보 제공을 넘어 교내 편의시설(카페, 서점, 휴게공간 등)의 정보도 통합 제공할 예정이다. 이를 통해 사용자는 하나의 애플리케이션에서 교내 모든 시설의 위치, 운영시간, 혼잡도, 재고 상태 등을 종합적으로 확인할 수 있을 것이다.

또한, QR 결제 기능이 학식뿐 아니라 교내 카페나 편의점에서도 적용될 수 있도록 범용화가 필요하다. 결제 내역 및 리뷰 기능은 AI 기반 추천 시스템과 연동되어, 사용자 개인의 선호 메뉴나 이용 패턴을 분석해 맞춤형 서비스를 제공할 계획이다.

더불어, 향후 서버 구조는 현재의 단일 Kubernetes Cluster에서 멀티 클러스터 구조로 확장되어, 트래픽 분산 및 안정성을 높일 것으로 예상된다. 데이터 수집 또한 실시간 센서 데이터 및 IoT 기반 인원 계수 시스템을 도입하여, 보다 정확한 혼잡도 예측 서비스를 구현할 예정이다.

4. Additional Information

4.1 Format

이 소프트웨어 요구사항 명세서(SRS)는 IEEE Recommendation(IEEE Recommended Practice for Software Requirements Specifications, IEEE-Std-830) 서식을 따라 제작되었다.